Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ITMO University

ПРАКТИЧЕСКАЯ РАБОТА

По дисциплине Контейнеризация и оркестрация

Тема работы Знакомство с экосистемой Docker

Обучающийся Ямалиева Альбина Азатовна

Факультет прикладной информатики

Группа K3320

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся	_____	_____	Ямалиева А.А.
	(дата)	(подпись)	(Ф.И.О.)
Руководитель	_____	_____	Аминов Н.С.
	(дата)	(подпись)	(Ф.И.О.)

СОДЕРЖАНИЕ

Введение 3

1. Верификация окружения и архитектуры 4

2. Первые образы 6

3. Жизненный цикл контейнера 9

4. Диагностика реального сервиса 11

Заключение 13

Введение

Цель работы – ознакомиться с экосистемой Docker, основными командами управления образами и контейнерами, изучить жизненный цикл контейнера, а также освоить базовые приёмы диагностики сервисов, запущенных в Docker. В ходе работы необходимо научиться запускать, удалять контейнеры, анализировать их состояния и логи, а также настраивать запуск сервиса PostgreSQL.

1. Верификация окружения и архитектуры

Номер ИСУ	472706
Персональный порт	5430 + последние 2 цифры ИСУ: 5430 + 06 = 5436
Персональный префикс	Фамилия латиницей в нижнем регистре: yamalieva

Перед началом работы проверим скачен ли Docker(Рисунок 1). Версия клиента: 29.8.2, версия сервера: 29.8.2. Команда *docker info --format "{{.OSType}}/{{.Architecture}}"* выводит тип операционной система, на которой работает Docker и архитектуру процессора.

```
qyalim@qy-redmi:~$ docker version --format "Client: {{.Client.Version}}, Server: {{.Server.Ver
sion}}"
Client: 29.8.2, Server: 29.8.2
qyalim@qy-redmi:~$ docker info --format "{{.OSType}}/{{.Architecture}}"
linux/x86_64
```

Рисунок 1 – Проверка Docker

Состояние локального хранилища представлено на рисунке 2. Локально хранится 5 образов, но активных на данный момент 0.

```
qyalim@qy-redmi:~$ docker system df --format "table {{.Type}}\t{{.TotalCount}}\t{{.Active}}\t{
{.Size}}"
TYPE            TOTAL     ACTIVE    SIZE
Images          5         0         1.05GB
Containers      0         0         0B
Local Volumes   0         0         0B
Build Cache     24        0         1.11GB
```

Рисунок 2 – Состояние локального хранилища

Выводим имя текущего пользвоателя, имя хоста машины, дату и время и версию Docker(Рисунок 3).

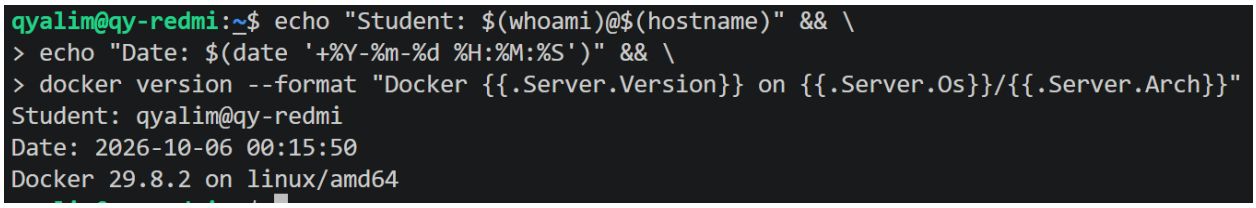


Рисунок 3 – Артефакты окружения

1. Что произойдёт, если запустить *docker system prune* без флагов? Какие данные будут потеряны?

docker system prune по умолчанию удалит неиспользуемые Docker объекты: остановленные контейнеры, неиспользуемые сети, dangling-образы и кэш сборки. Перед удалением Docker попросит подтверждение. Тома по умолчанию не удаляются, для этого нужен отдельный флаг --volumes.

2. Почему команда *docker version* может завершиться ошибкой, даже если docker установлен

Это может произойти, если Docker daemon не запущен или клиент не может к нему подключиться. То есть сам Docker может быть установлен, но серверная часть Docker недоступна.

2. Первые образы

Создаем контейнер, который автоматически удаляется(Рисунок 4).

```
qyalim@qy-redmi:~$ docker run --rm hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

Рисунок 4 - Пример

1. Откуда загружается образ (полное имя репозитория и тег, например docker.io/library/hello-world:latest)

Образ hello-world загружается из Docker Hub. Полное имя образа: docker.io/library/hello-world:latest

Это соответствует тому, что в выводе указано: Docker daemon скачал образ hello-world из Docker Hub.

2. Какой процесс выполняется внутри контейнера (подсказка: см. вывод)

Внутри контейнера запускается исполняемый файл: /hello

Он выводит сообщение Hello from Docker! и после этого завершается, поэтому контейнер тоже сразу останавливается.

Исследуем минимальный образ на базе Alpine Linux(Рисунок 5). PRETTY_NAME – поле из файла /etc/os-release, в котором хранится название ОС, понятное человеку. Размер данного образа представлен на рисунке 6.

```
qyalim@qy-redmi:~$ docker run --rm alpine:3.18 cat /etc/os-release
Unable to find image 'alpine:3.18' locally
3.18: Pulling from library/alpine
44cf07d57ee4: Pull complete
431d356fe850: Download complete
54d130656ede: Download complete
Digest: sha256:de0eb0b3f2a47ba1eb89389859a9bd88b28e82f5826b6969ad604979713c2d4f
Status: Downloaded newer image for alpine:3.18
NAME="Alpine Linux"
ID=alpine
VERSION_ID=3.18.12
PRETTY_NAME="Alpine Linux v3.18"
HOME_URL="https://alpinelinux.org/"
BUG_REPORT_URL="https://gitlab.alpinelinux.org/alpine/aports/-/issues"
```

Рисунок 5 – Запуск контейнера на базе Alpine Linux

```
qyalim@qy-redmi:~$ docker images alpine:3.18 --format "{{.Size}}"
11.5MB
```

Рисунок 6 – Размер образа

Далее мы создали два контейнера на базе Alpine Linux. Оба контейнера работали 5 минут, а второй также вывел сообщение(Рисунок 7)

```
qyalim@qy-redmi:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS     NAMES
504ebd286cac   alpine:3.18    "sh -c 'echo 'Hello …"   11 seconds ago   Up 11 seconds             yamalieva-alpine-2
703b66966cf1   alpine:3.18    "sleep 300"              17 seconds ago   Up 17 seconds             yamalieva-alpine-1
```

Рисунок 7 – Создание двух alpine

1. Почему образ hello-world завершается сразу после запуска, а alpine:3.18 sleep 300 работает 5 минут?

hello-world завершается сразу, потому что его главный процесс /hello выводит сообщение и заканчивает работу. Контейнер живёт только пока работает его главный процесс. В alpine-1 главным процессом является sleep 300, поэтому контейнер остаётся запущенным 300 секунд.

2. Что произойдёт, если убрать флаг --rm из команды docker run --rm hello-world? Как удалить «забытые» контейнеры?

Если убрать флаг --rm, контейнер после завершения не удалится автоматически, а останется в системе в состоянии Exited. Флаг --rm как раз отвечает за автоматическое удаление контейнера после его остановки.

Удалить конкретный контейнер можно через *docker r, <name/ID>* или все остановленные контейнеры через *docker container prune.*

3. Жизненный цикл контейнера

Рассмотрим разные жизненные циклы контейнера: создание, запуск, остановка, удаление(Рисунки 8-13). После выполнения каждой команды фиксируем состояние командой *docker ps -a --filter "name=<фамилия>-nginx" --format "table {{.Names}}\t{{.Status}}\t{{.CreatedAt}}"* .

```
qyalim@qy-redmi:~$ docker create --name yamalieva-nginx nginx:alpine
76a5eb273dd3a00a10319a74966daa023a425d048df2cb56726d3fbd08536b7c
qyalim@qy-redmi:~$ docker ps -a --filter "name=yamalieva-nginx" --format "table {{.Names}}\t{{.
NAMES             STATUS    CREATED AT
yamalieva-nginx   Created   2026-10-06 12:53:35 +0300 MSK
```

Рисунок 8 – Создание контейнера без запуска

```
qyalim@qy-redmi:~$ docker start yamalieva-nginx
yamalieva-nginx
qyalim@qy-redmi:~$ docker ps -a --filter "name=yamalieva-nginx" -
NAMES             STATUS         CREATED AT
yamalieva-nginx   Up 1 second    2026-10-06 12:53:35 +0300 MSK
```

Рисунок 9 - Запуск

```
qyalim@qy-redmi:~$ docker stop yamalieva-nginx
yamalieva-nginx
qyalim@qy-redmi:~$ docker ps -a --filter "name=yamalieva-nginx" --format "t
NAMES             STATUS                     CREATED AT
yamalieva-nginx   Exited (0) 1 second ago    2026-10-06 12:53:35 +0300 MSK
```

Рисунок 10 – «Мягкая» остановка

```
qyalim@qy-redmi:~$ docker restart yamalieva-nginx
yamalieva-nginx
qyalim@qy-redmi:~$ docker ps -a --filter "name=yamalieva-nginx" --fo
NAMES             STATUS         CREATED AT
yamalieva-nginx   Up 2 seconds   2026-10-06 12:53:35 +0300 MSK
```

Рисунок 11 - Перезапуск

```
qyalim@qy-redmi:~$ docker kill yamalieva-nginx
yamalieva-nginx
qyalim@qy-redmi:~$ docker ps -a --filter "name=yamalieva-nginx" --format "tabl
NAMES             STATUS                      CREATED AT
yamalieva-nginx   Exited (137) 1 second ago   2026-10-06 12:53:35 +0300 MSK
```

Рисунок 12 – «Жесткая» остановка

```
qyalim@qy-redmi:~$ docker rm yamalieva-nginx
yamalieva-nginx
qyalim@qy-redmi:~$ docker ps -a --filter "name=yamalieva-nginx"
NAMES     STATUS    CREATED AT
qyalim@qy-redmi:~$
```

Рисунок 13 - Удаление

1. Какой сигнал отправляет *docker stop* по умолчанию?

docker stop по умолчанию отправляет контейнеру сигнал SIGTERM. Это «мягкая» остановка: процесс получает возможность корректно завершиться, закрыть соединения и сохранить данные.

2. Почему *docker kill* может привести к потере данных в контейнере с БД?

docker kill отправляет SIGKILL, то есть процесс завершается мгновенно и не успевает выполнить корректное завершение работы. Для контейнера с БД это опасно тем, что могут не успеть записаться данные на диск или корректно завершиться текущие операции, из-за чего возможна потеря данных.

3. Что произойдет, если выполнить *docker rm* для запущенного контейнера? Какой флаг нужен для принудительного удаления?

Если выполнить *docker rm <name/ID>* для запущенного контейнера, то Docker выдаст ошибку и не удалит его. Сначала контейнер нужно остановить.

Для принудительного удаления используется *docker rm -f <name/ID>*

Флаг -f принудительно останавливает и удаляет контейнер.

4. Диагностика реального сервиса

Запустим postgres с ошибкой, ошибка является в том, что не был указан пароль(Рисунок 14)

```
qyalim@qy-redmi:~$ docker run -d \
> --name yamalieva-pg-broken \
> -p 5436:5432 \
> postgres:15
Unable to find image 'postgres:15' locally
15: Pulling from library/postgres
ecc510c1e359: Pull complete
b3db397181ff: Pull complete
8bc3c2e35349: Pull complete
2260c4957d6a: Pull complete
9147e7910f0b: Pull complete
2fc81bab6318: Pull complete
070873950b18: Pull complete
284f94de039c: Pull complete
554af8d1cb5e: Pull complete
c503228ba229: Pull complete
f410f7e4b5bb: Pull complete
1334c6144a55: Pull complete
4b5dc4352899: Pull complete
79f55b6c0eb9: Pull complete
44fe08796ef2: Download complete
f8b7812b3ccb: Download complete
Digest: sha256:7e2070cf6ad06fb3cbbd141b1bafbb7fd5bb63e2b6001e6daf34448eb555e4b3
Status: Downloaded newer image for postgres:15
b3a57968cf1f63c9d14480be5d230f25e23f67c74bcfe9632873e3c988ada06f
```

Рисунок 14 – Запуск с ошибкой

Проведем диагностику: проверим статус контейнера, последние логи, код завершения(Рисунки 15-16).

```
qyalim@qy-redmi:~$ docker ps -a --filter "name=yamalieva-pg"
CONTAINER ID   IMAGE         COMMAND                  CREATED              STATUS                          PORTS     NAMES
b3a57968cf1f   postgres:15   "docker-entrypoint.s…"   About a minute ago   Exited (1) About a minute ago             yamalieva-pg-broken
qyalim@qy-redmi:~$ docker logs --timestamps yamalieva-pg-broken 2>&1 | tail -20
2026-10-06T09:59:29.701049189Z Error: Database is uninitialized and superuser password is not specified.
2026-10-06T09:59:29.701137098Z        You must specify POSTGRES_PASSWORD to a non-empty value for the
2026-10-06T09:59:29.701143899Z        superuser. For example, "-e POSTGRES_PASSWORD=password" on "docker run".
2026-10-06T09:59:29.701148199Z
2026-10-06T09:59:29.701152500Z        You may also use "POSTGRES_HOST_AUTH_METHOD=trust" to allow all
2026-10-06T09:59:29.701156800Z        connections without a password. This is *not* recommended.
2026-10-06T09:59:29.701160601Z
2026-10-06T09:59:29.701164401Z        See PostgreSQL documentation about "trust":
2026-10-06T09:59:29.701168902Z        https://www.postgresql.org/docs/current/auth-trust.html
```

Рисунок 15 - Диагностика

```
qyalim@qy-redmi:~$ docker inspect yamalieva-pg-broken --format='{{.State.ExitCode}}'
1
```

Рисунок 16 - Диагностика

Исправленный запуск представлен на рисунке 17.

```
qyalim@qy-redmi:~$ docker run -d \
> --name yamalieva-pg-fixed \
> -e POSTGRES_PASSWORD=Pass_5436 \
> -e POSTGRES_USER=user_472706 \
> -p 5436:5432 \
> postgres:15
c6e9dc8787a4ac05ab10a7498623dde3d30ad5c763e1faf865b03197e066a0fa
```

Рисунок 17 – Запуск

Проверяем работоспособность и выполняем запрос к БД(Рисунок 18).

```
qyalim@qy-redmi:~$ sleep 15
qyalim@qy-redmi:~$ docker exec yamalieva-pg-fixed psql -U user_472706 -c "SELECT version();"
                                                 version
---------------------------------------------------------------------------------------------------------
 PostgreSQL 15.19 (Debian 15.19-1.pgdg13+2) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
(1 row)
```

Рисунок 18 - Проверка

В конце выводим логин, имя хоста, время и статус контейнера(Рисунок 19).

```
=== Практика №1: 2026-10-06 13:08:10 ===
Студент: qyalim@qy-redmi
NAMES                STATUS         PORTS
yamalieva-pg-fixed   Up 2 minutes   0.0.0.0:5436->5432/tcp, [::]:5436->5432/tcp
```

Рисунок 19 – «Финальный» артефакт

Заключение

В ходе практической работы были изучены основные команды Docker и принципы работы контейнеров. Была выполнена проверка окружения и локального хранилища Docker, запущены контейнеры на основе образов hello-world и alpine, исследовано их поведение и жизненный цикл. Также были рассмотрены различия между мягкой и принудительной остановкой контейнеров, а на примере PostgreSQL выполнена диагностика ошибки запуска по логам и коду завершения, после чего сервис был корректно запущен с необходимыми параметрами. В результате были получены базовые навыки работы с Docker и диагностики контейнеров.